MACQUARIE University

COMP8221 Advanced Machine Learning

Graph Convolutional Network

Dr. Qiongkai Xu, School of Computing, qiongkai.xu@mq.edu.au



Outline

- Graph Convolutional Networks
- Permutation Invariant / Equivalent
- General Graph NN framework
- GraphSAGE
- Graph Attention Networks

Recap: Graph Structured Data

- Image

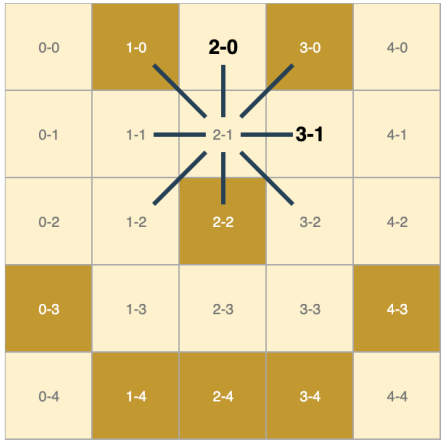
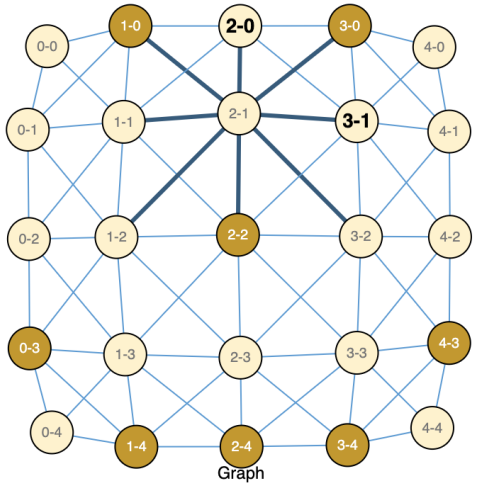
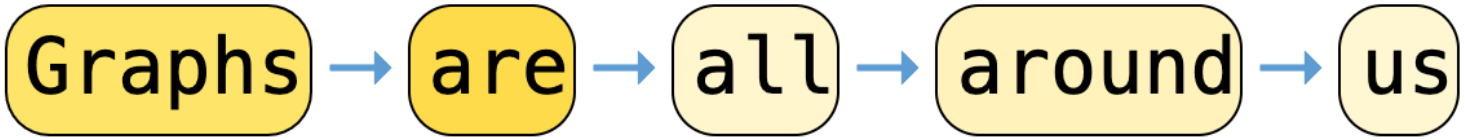


Image Pixels



Graph

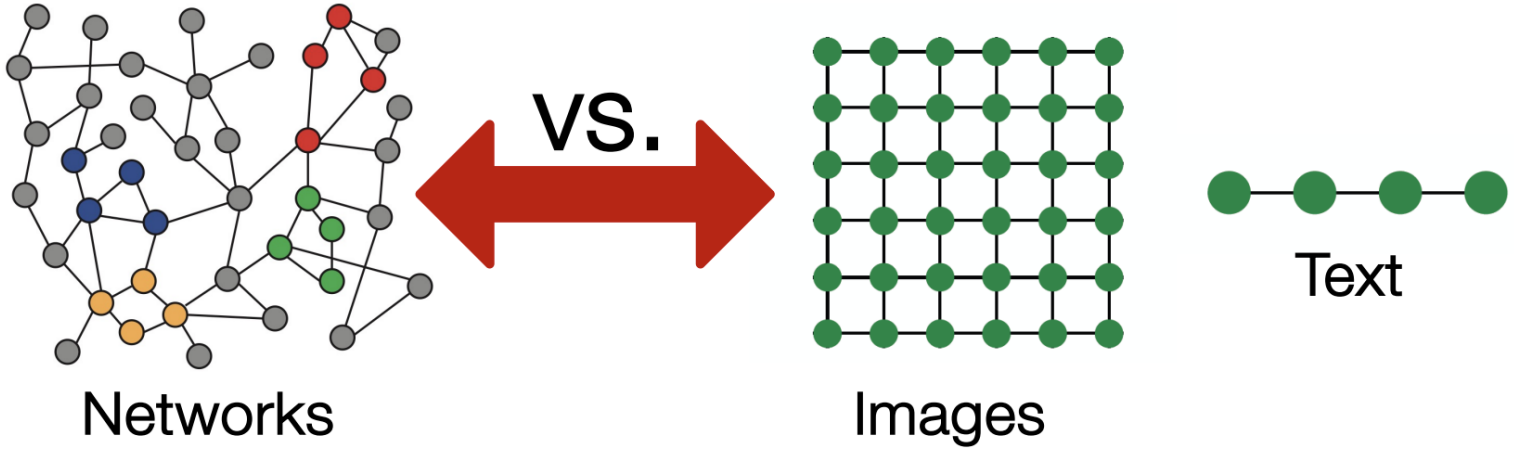
- Text/Speech



Recap: Graph Structured Data

MACQUARIE University

Networks are far more complex.



- Arbitrary size and complex topological structure.
- No fixed node ordering or reference point.
- Often dynamic and have multimodal features

Today: Deep Graph Encoders

MACQUARIE University

Deep Graph Encoders:

Deep learning methods based on graph neural networks (GNNs)

$$Enc(v) = \text{Multiple layers of non-linear transformations based on graph structure}$$

Graph Setup

MACQUARIE University

Assume we have a graph:

- $G = (V, E)$
- V is the vertex set
- E is the edge set (undirected edge & directed edge)
- $\boldsymbol{A}$ is the Adjacency matrix
- $\boldsymbol{X} \in R^{|V| \times m}$ is a matrix of node features
- v represents a node in V
- $N(v)$ is the set of neighbors of v

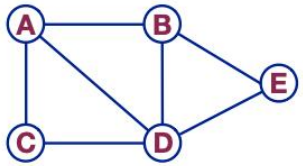
- Examples of node features:

Social networks: User profile, User image, …

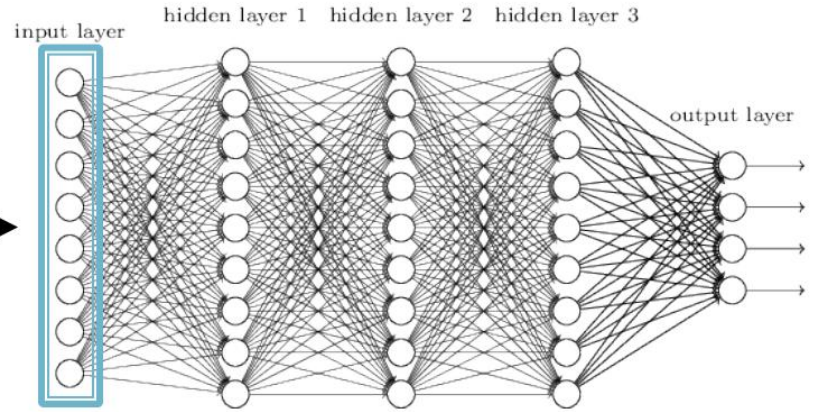
Biological networks: Gene expression profile, Gene functional information, …

A Naïve Approach

- Join adjacency matrix and features
- Feed them into a neural network



	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0



?

- Issues?

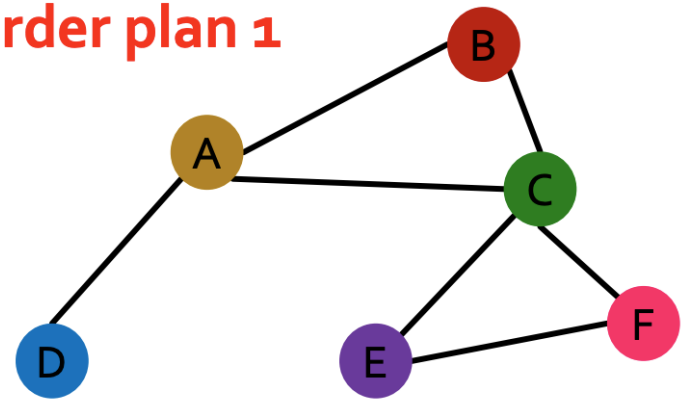
A Naïve Approach

Graph does not have a canonical order of the nodes!

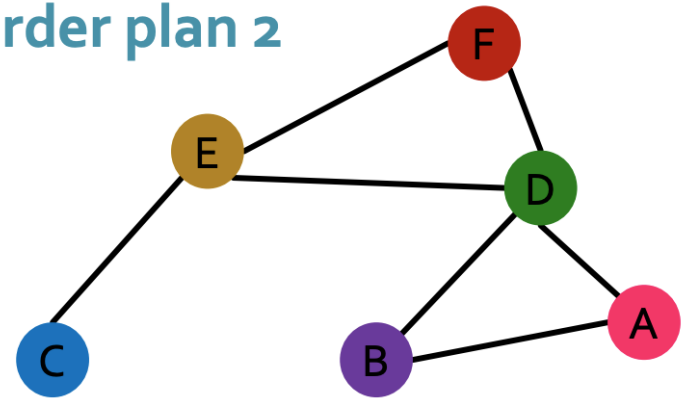
Consider this example:
Same graph with different ordering

The naïve approach would have different inputs for the same graph

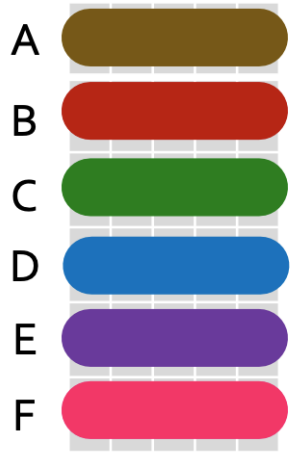
Order plan 1



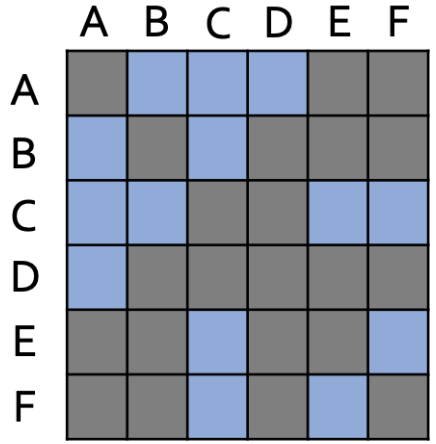
Order plan 2



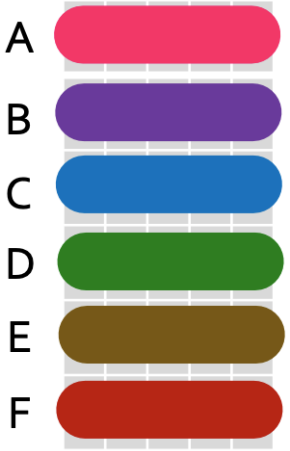
Node features X_1



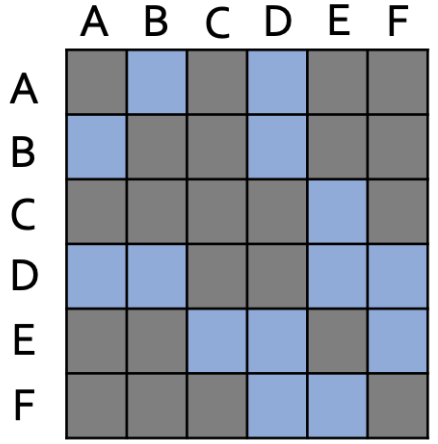
Adjacency matrix A_1



Node features X_2



Adjacency matrix A_2

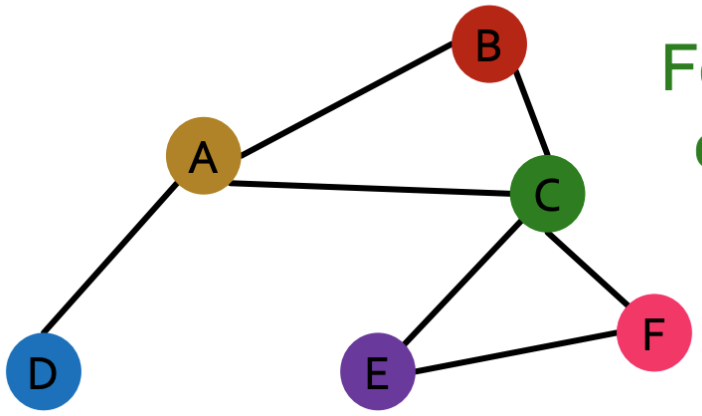


Permutation Invariance

- Graph and node representations should be the **same** for different order plan.
- That is, we learn a function f that maps a Graph G to a vector R^d for embedding that

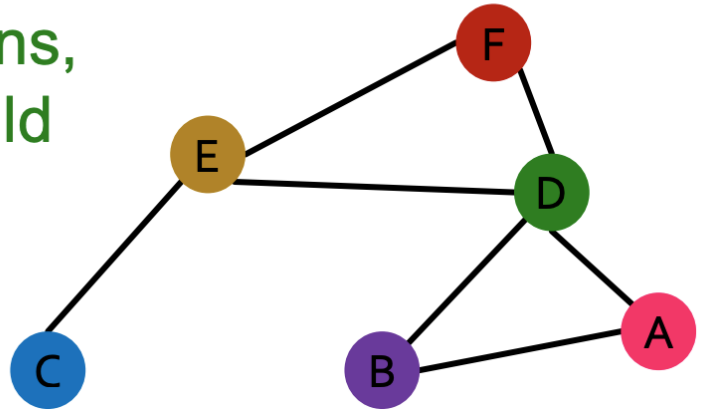
$$f(\boldsymbol{A}_1, \boldsymbol{X}_1) = f(\boldsymbol{A}_2, \boldsymbol{X}_2)$$

Order plan 1: $\boldsymbol{A}_1, \boldsymbol{X}_1$



For two order plans, output of f should be the same!

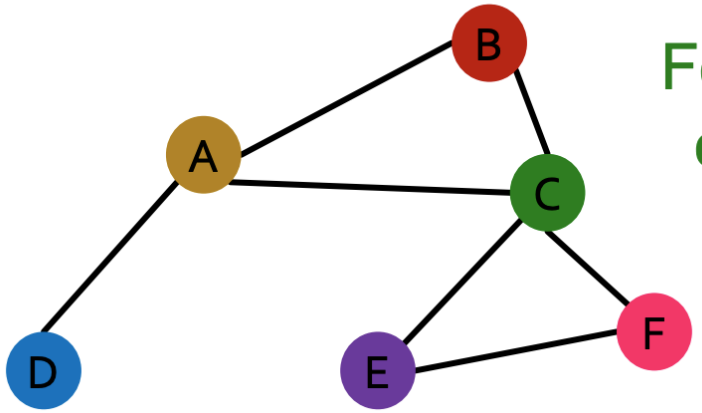
Order plan 2: $\boldsymbol{A}_2, \boldsymbol{X}_2$



Permutation Invariance

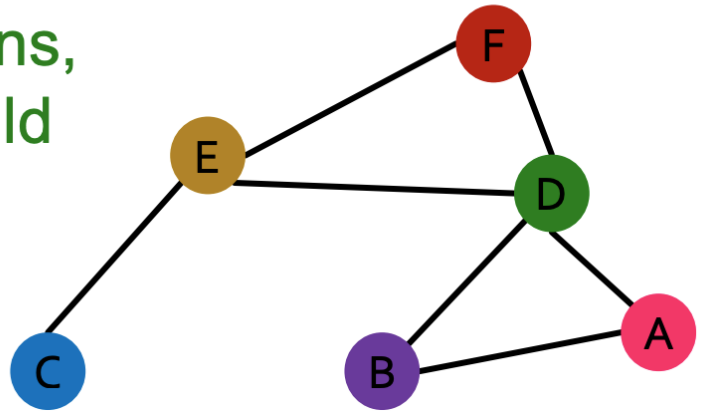
- For a graph with $|V|$ nodes, there are $|V|!$ different order plans.
- If $f(\boldsymbol{A}_i, \boldsymbol{X}_i) = f(\boldsymbol{A}_j, \boldsymbol{X}_j)$ for any order plan i and j, we formally say that f is a **permutation invariant function**.

Order plan 1: $\boldsymbol{A}_1, \boldsymbol{X}_1$



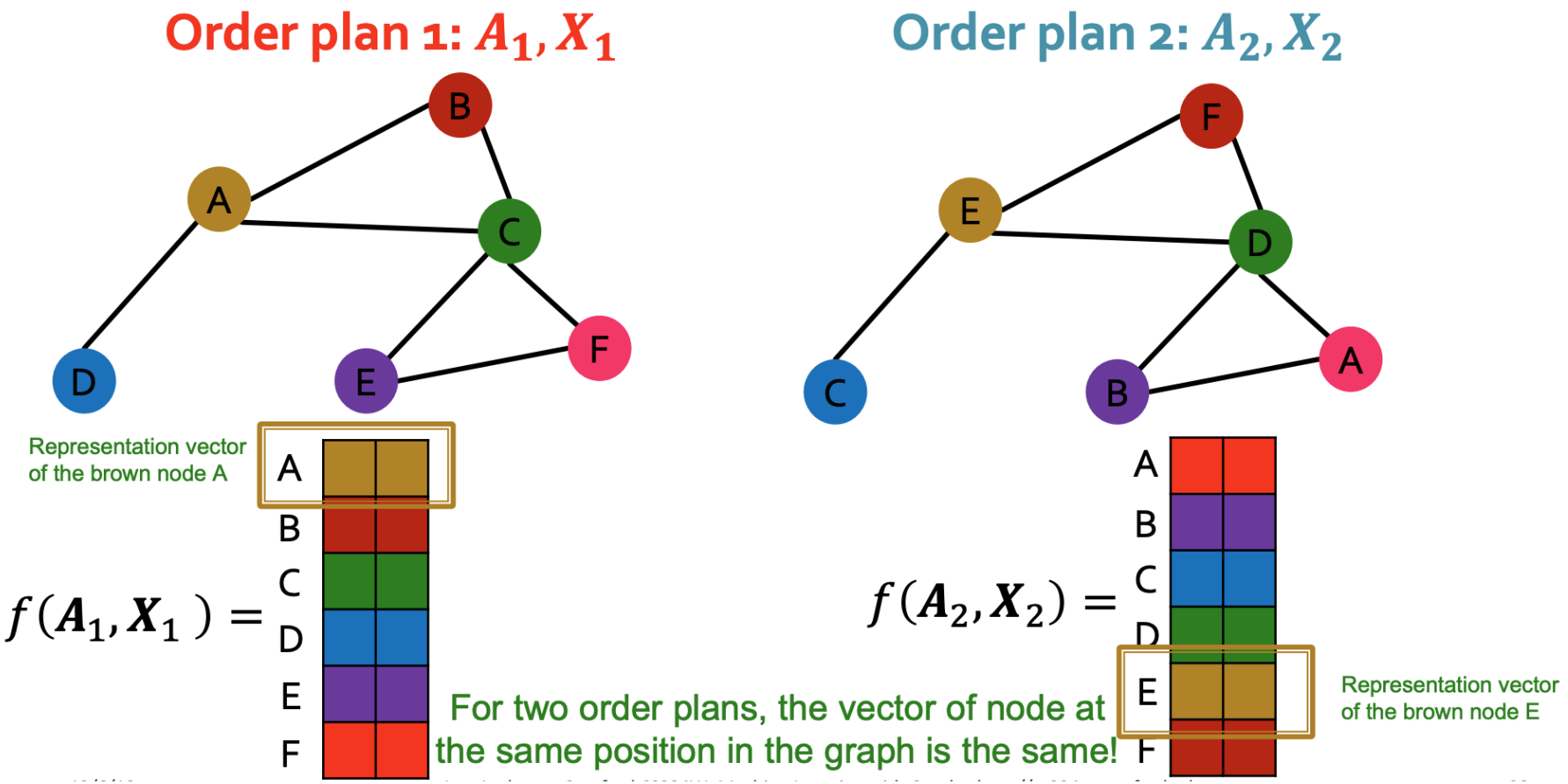
For two order plans, output of f should be the same!

Order plan 2: $\boldsymbol{A}_2, \boldsymbol{X}_2$



Permutation Equivariance

- Consider we learn a function f that maps nodes in G to a matrix $R^{|V|\times d}$.



Permutation Equivariance

MACQUARIE University

- Consider we learn a function f that maps nodes in G to a matrix $R^{|V|\times d}$.
- If the output vector of a node at the same position in the graph remains unchanged for any order plan, we say f **is permutation equivariant**.

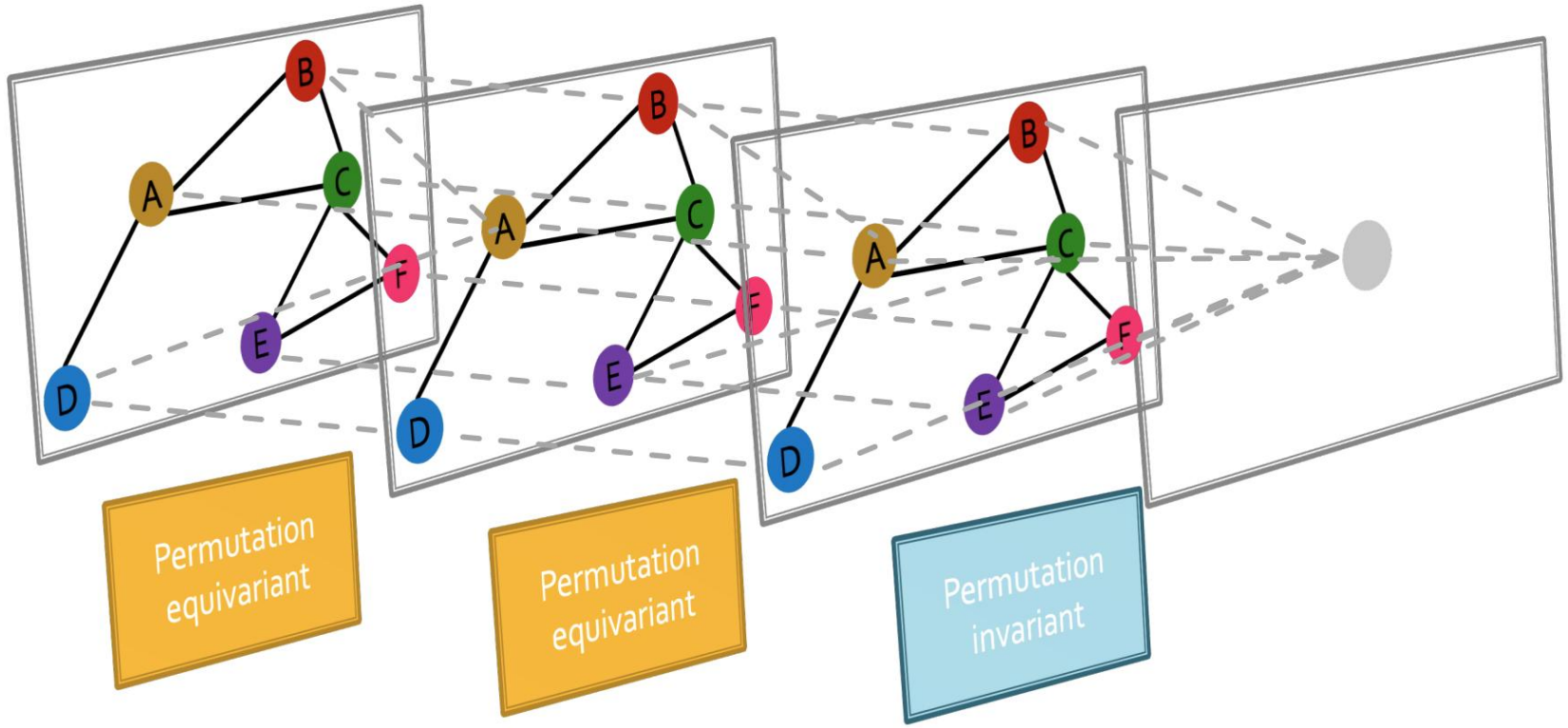
In summary,

- **Permutation Invariant**: Map a graph to a vector, if we permute the input, output stays the **same**
- **Permutation Equivariant**: Map a graph to a matrix, if we permute the input, output **permutes accordingly.**

Graph Neural Networks

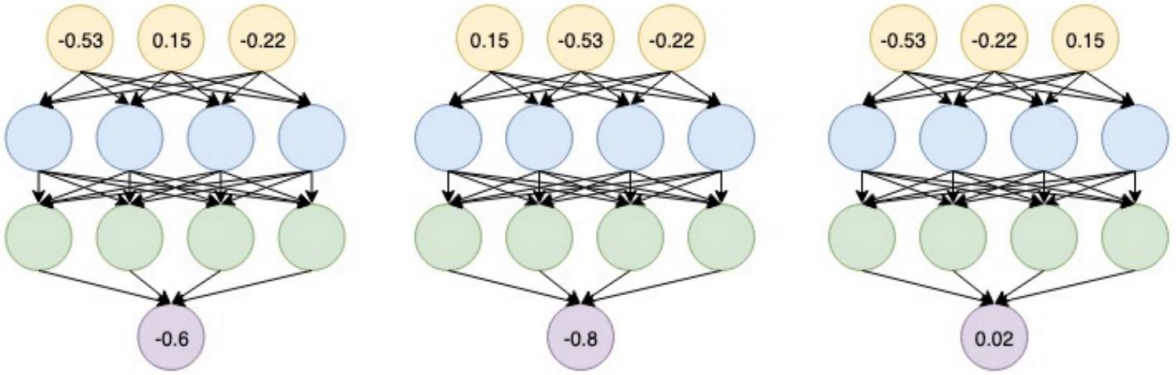
MACQUARIE University

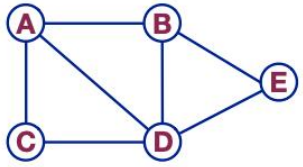
- Graph neural networks consist of multiple permutation equivariant / invariant functions.



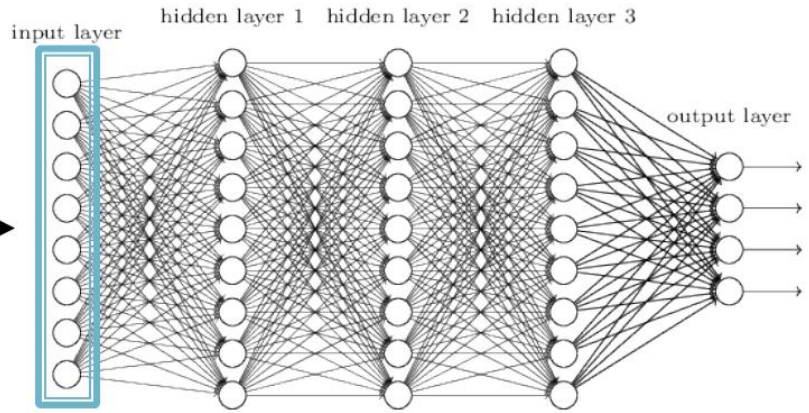
Why naïve approach fails.

- Are classical neural network architectures permutation invariant / equivariant?





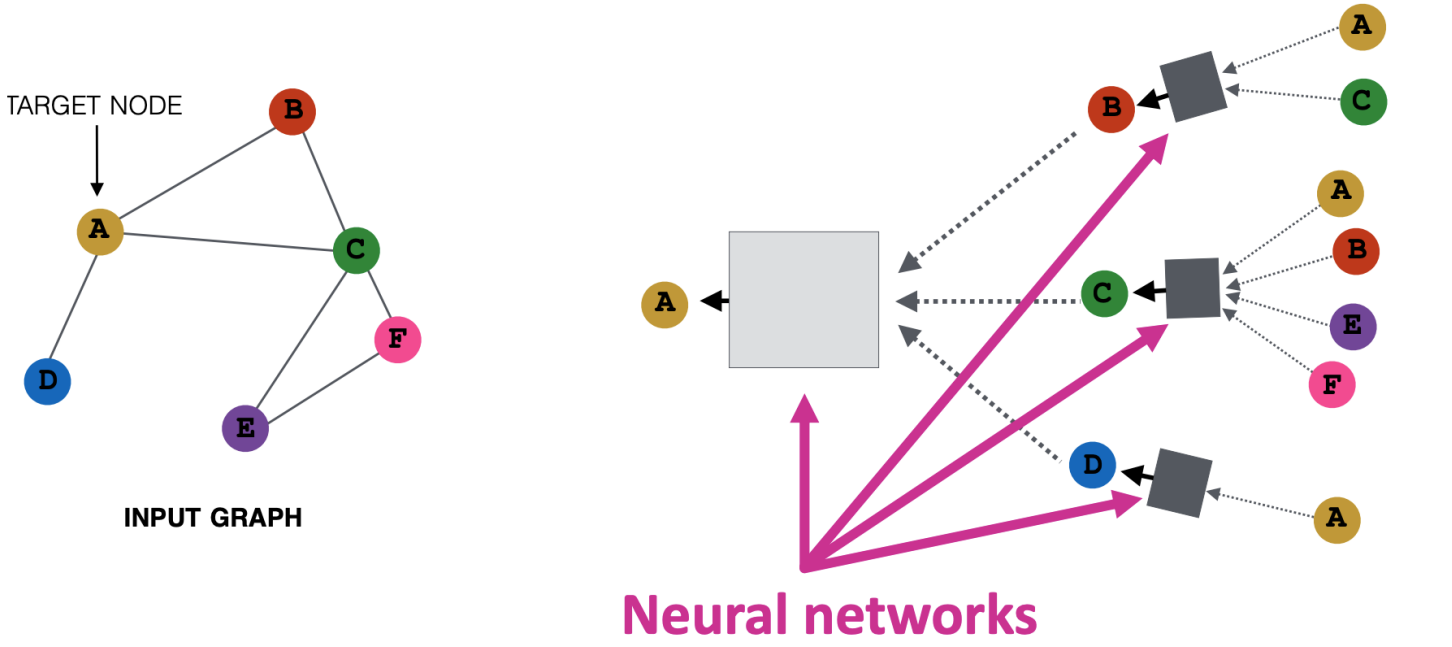
	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0



?

Graph Convolutional Networks

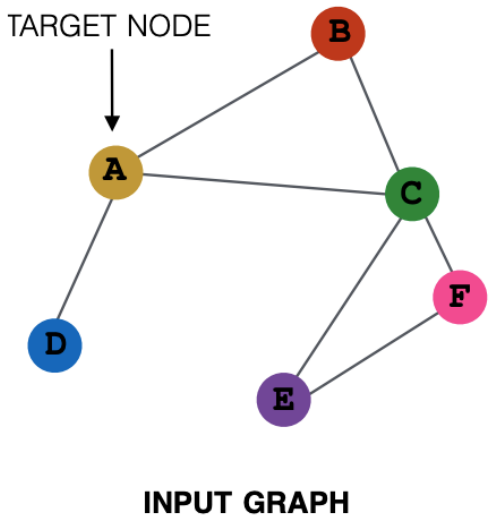
- Design graph neural networks that are permutation invariant / equivariant.
- **Key idea**: Generate node embeddings based on **local network neighborhood**.
- **Intuition**: Nodes aggregate information from their neighbours using neural networks



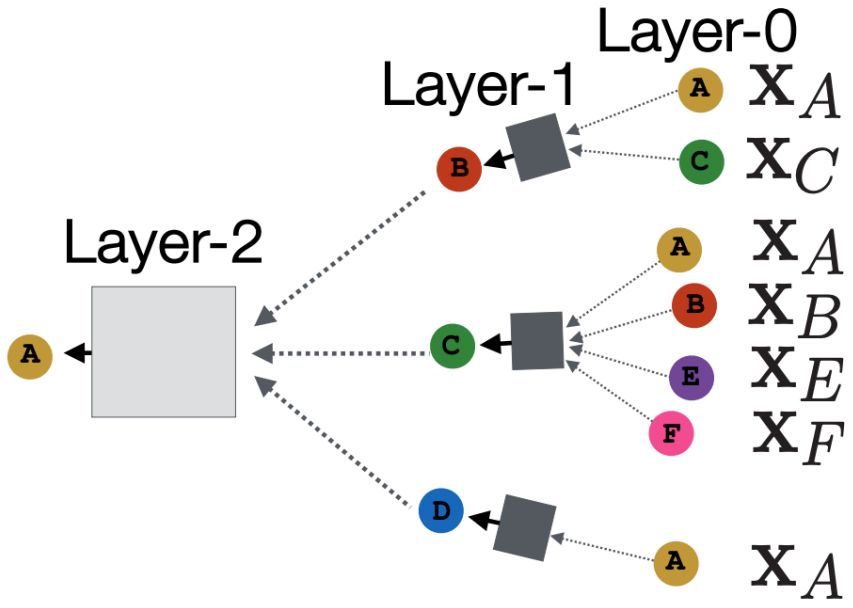
Graph Convolutional Networks

MACQUARIE University

- Model can be of arbitrary length
- Nodes have embeddings at each layer
- Layer-0 embedding of node v is its input feature
- Layer-k embedding gets information from nodes that are k hops away



INPUT GRAPH



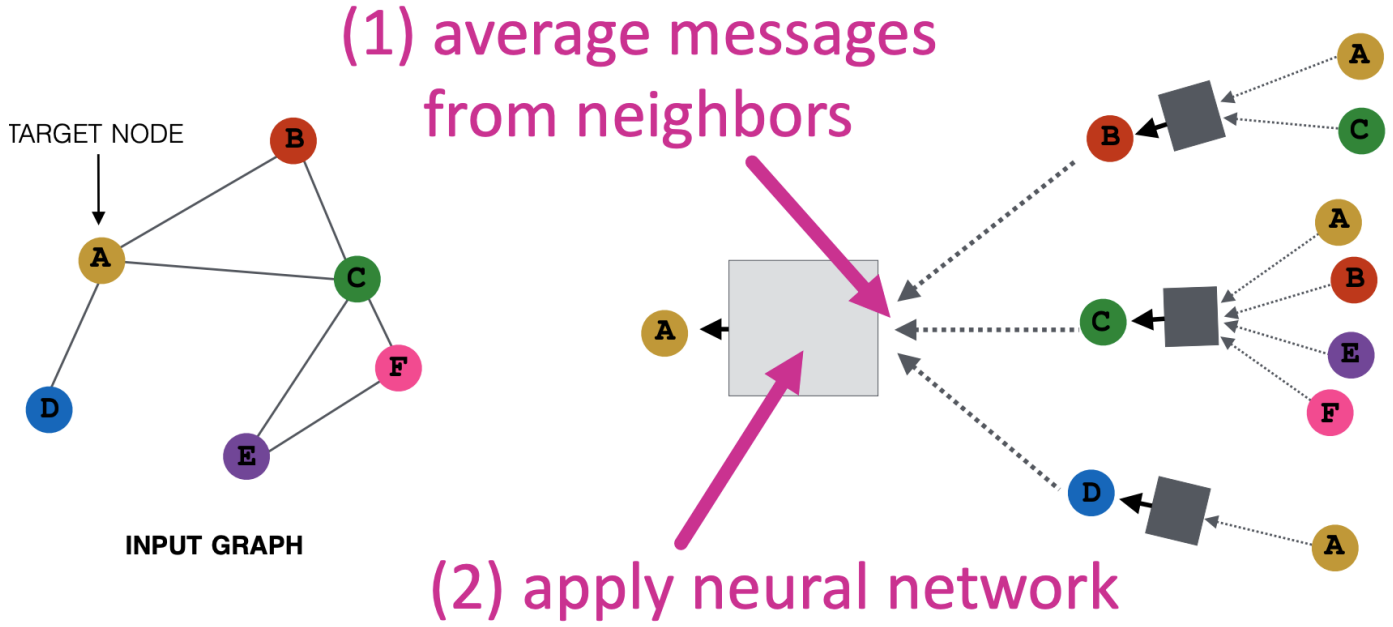
Idea: Neighbourhood Aggregation

MACQUARIE University

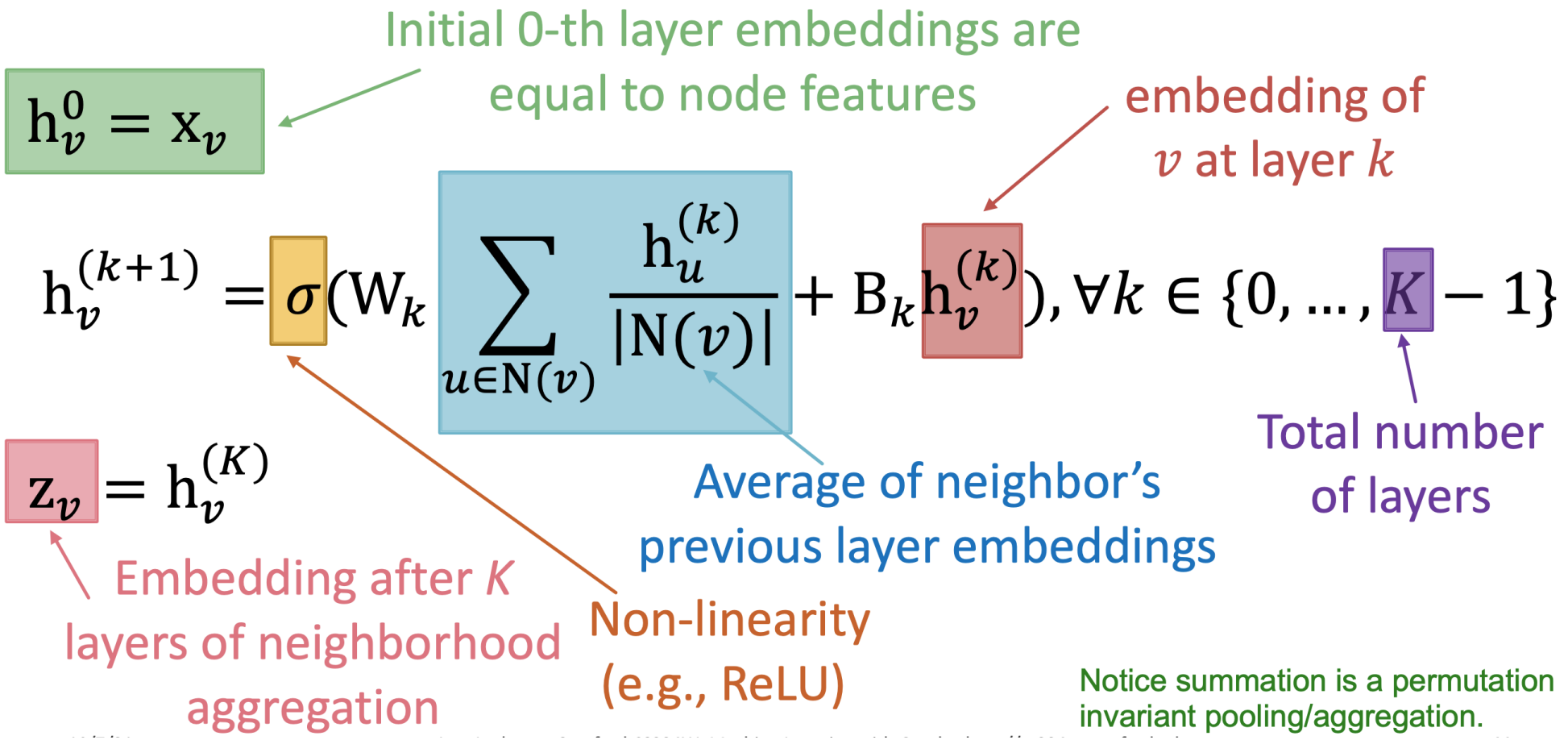
What is in the box?

Basic approach:

- 1) Average information from neighbors
- 2) apply a neural network.

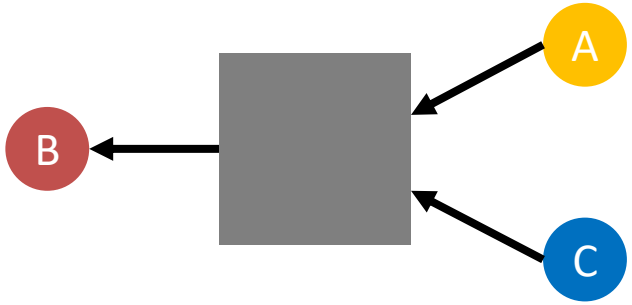


Neighbourhood Aggregation



Neighbourhood Aggregation

For example,



Initial 0-th layer embedding is its input feature X_B

$$h_B^0 = X_B$$

Non-linearity(e.g. ReLU)

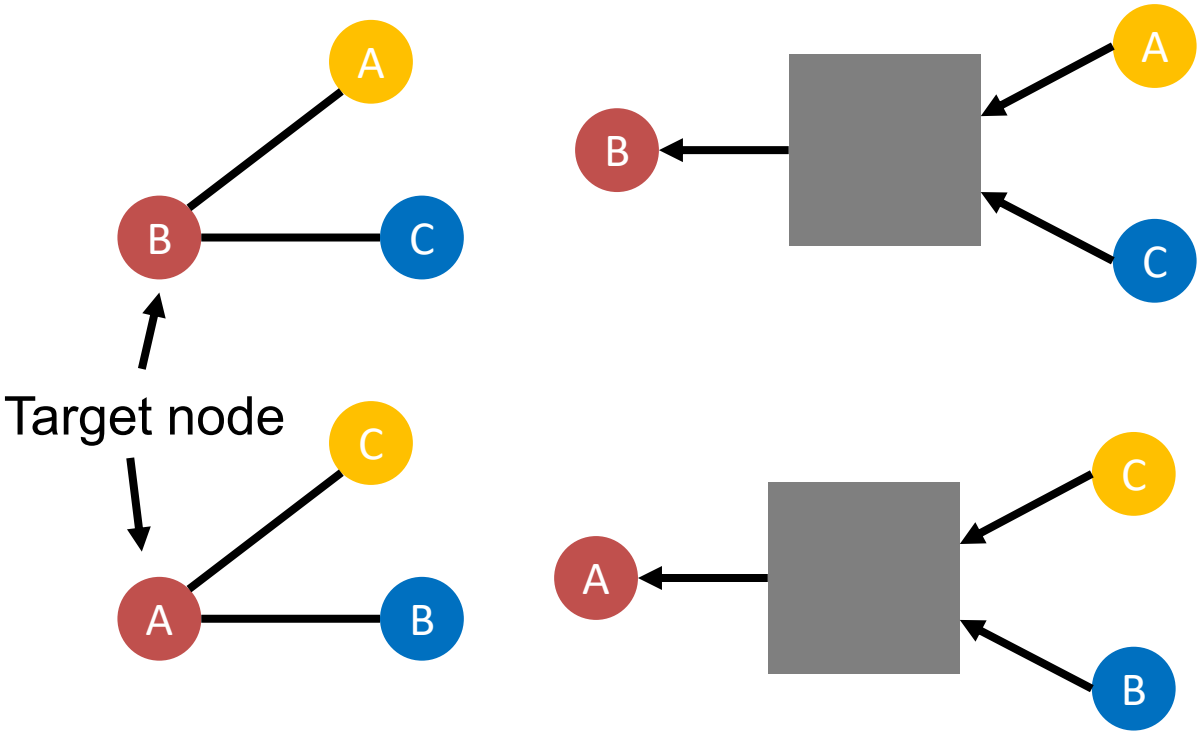
Self-transformation

$$h_B^1 = \sigma(W_0(\frac{h_A^0 + h_C^0}{2}) + B_0 h_B^0)$$

Average of B's neighbors' previous layer embedding

GCN: Invariance and Equivariance

Given a node, the GCN that computes its embedding is **permutation invariant** as average of neighbors' previous layer embeddings is **permutation invariant**.



$$h_B^1 = \sigma\left(W_0\left(\frac{h_A^0 + h_C^0}{2}\right) + B_0 h_B^0\right)$$

The Same

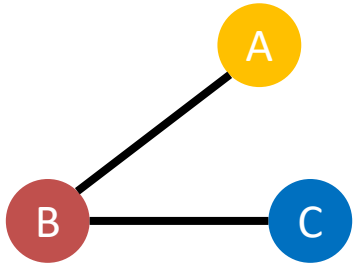
$$h_A^1 = \sigma\left(W_0\left(\frac{h_C^0 + h_B^0}{2}\right) + B_0 h_A^0\right)$$

Two order plans, colors represent input feature for each node

GCN: Invariance and Equivariance

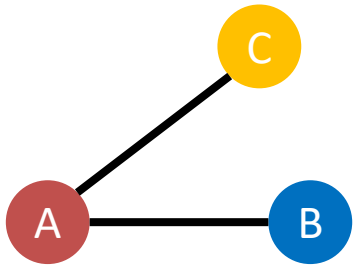
MACQUARIE University

Considering all nodes in a graph, GCN computation is **permutation equivariant.**



We know from previous slides that, when considering **single individual node**, GCN is **permutation invariant** as **output stays the same** when we change the order plan.

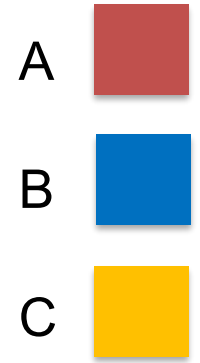
h_B^1 in order plan1 $=$ h_A^1 in order plan 2 (red),
h_A^1 in order plan1 $=$ h_C^1 in order plan 2 (yellow),
h_c^1 in order plan1 $=$ h_B^1 in order plan 2 $(Blue)$,



Order plan 1
GCN output:



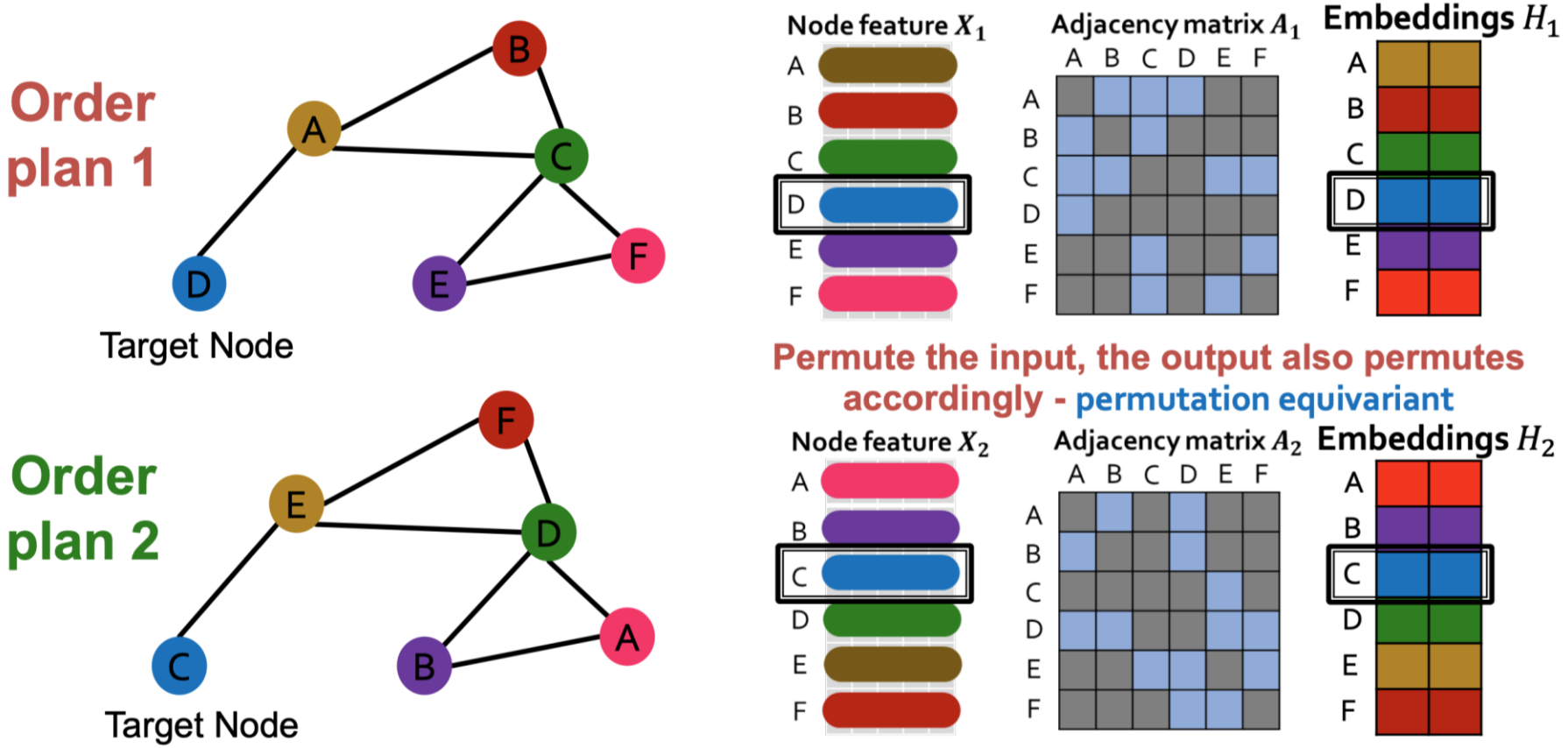
Order plan 2
GCN output:



Considering all nodes, when we change the order plan(permute the input), **output permutes** accordingly.

GCN: Invariance and Equivariance

Considering all nodes in a graph, GCN computation is **permutation equivariant.**



GCN: Invariance and Equivariance



MACQUARIE University

Considering all nodes in a graph, GCN computation is **permutation equivariant.**

Detailed reasoning:

1. The rows of **input node features** and **output embeddings** are aligned
2. We know computing the embedding of **a given node** with GCN is **invariant**.
3. So, after permutation, the location of a given node in the input node feature matrix is changed, and the the output embedding of a given node stays the same (the colors of node feature and embedding are matched)

This is permutation equivariant.

Training GCN

MACQUARIE University

Model parameters:

Trainable weight matrices
(i.e., what we learn)

$$h_v^{(0)} = x_v$$

$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \forall k \in \{0..K-1\}$$

$$z_v = h_v^{(K)}$$

Final node embedding

We can feed these **embeddings into any loss function** and run SGD to **train the weight parameters**

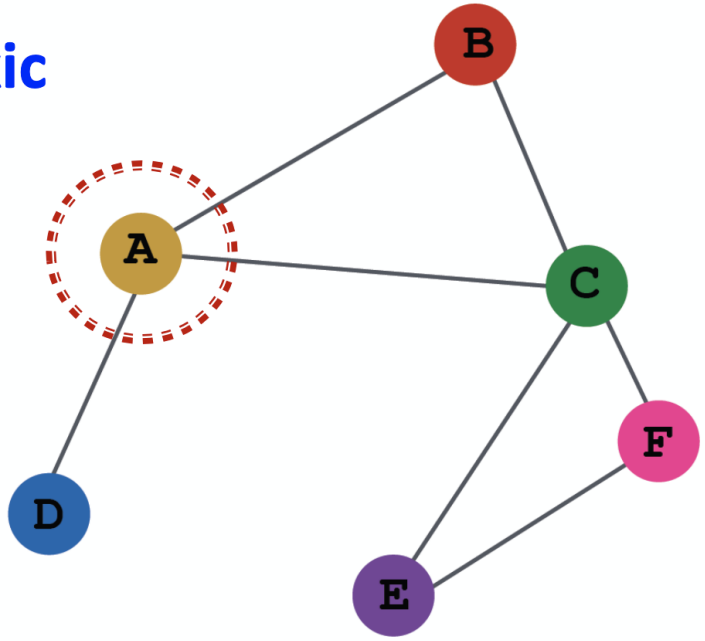
h_v^k: the hidden representation of node v at layer k

- W_k: weight matrix for neighborhood aggregation
- B_k: weight matrix for transforming hidden vector of self

Supervised Training

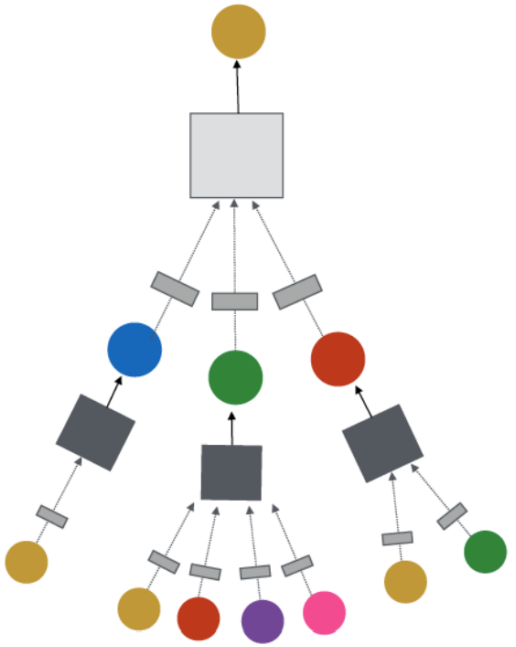
Directly train the model for a supervised task (e.g., node classification)

Safe or toxic drug?



E.g., a drug-drug interaction network

Safe or toxic drug?



Supervised Training

Use cross entropy loss:

$$CE\big(y, f(x)\big) = -\sum_{i=1}^{C} (y_i \log f_\theta(x)_i)$$

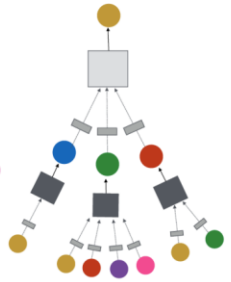
y_i , $f_\theta(x)_i$ are the actual and predicted values of the i-th class

$$\mathcal{L} = -\sum_{v \in V} y_v \log(\sigma(z_v^T \theta)) + (1 - y_v)\log(1 - \sigma(z_v^T \theta))$$

Encoder output: node embedding

Classification weights

Node class label (binary classification safe/toxic)



Safe or toxic drug?

Unsupervised Training

MACQUARIE University

One possible idea:

"Similar" nodes have similar embeddings

Again, use cross entropy loss

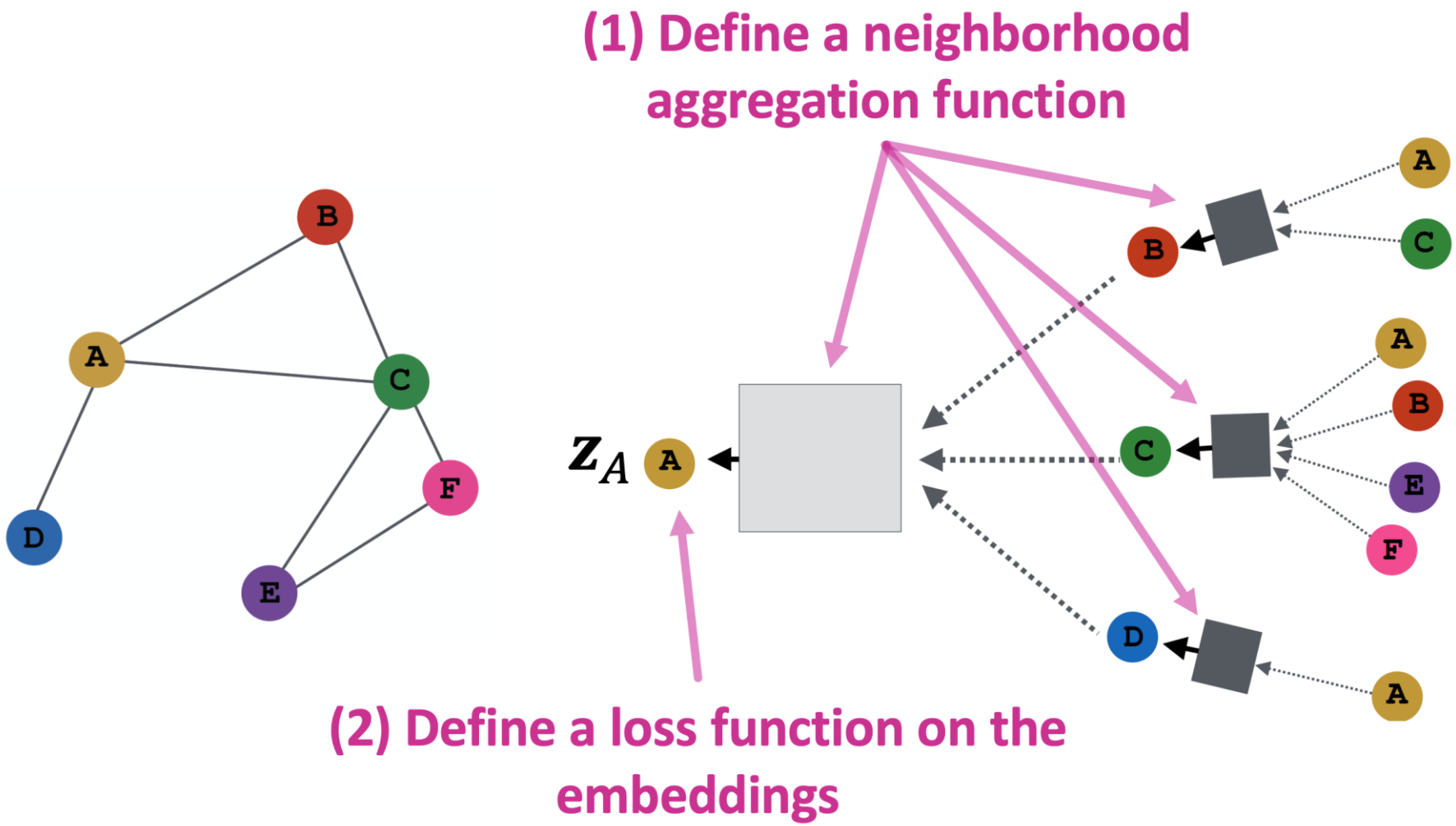
$$L = \sum_{z_u, z_v} CE(y_{u,v}, DEC(z_u, z_v))$$

where $y_{u,v} = 1$ when node u and v are similar

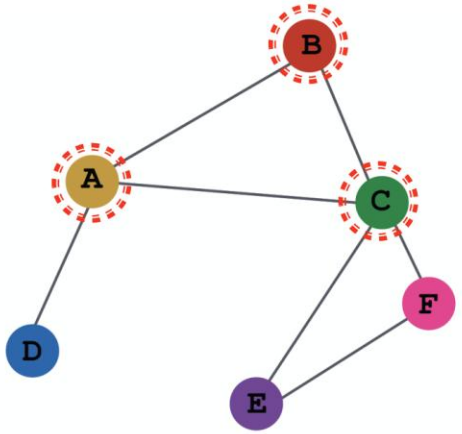
z_u, z_v are the node embeddings

$DEC(\cdot, \cdot)$ is the dot product

GNN Model Overview

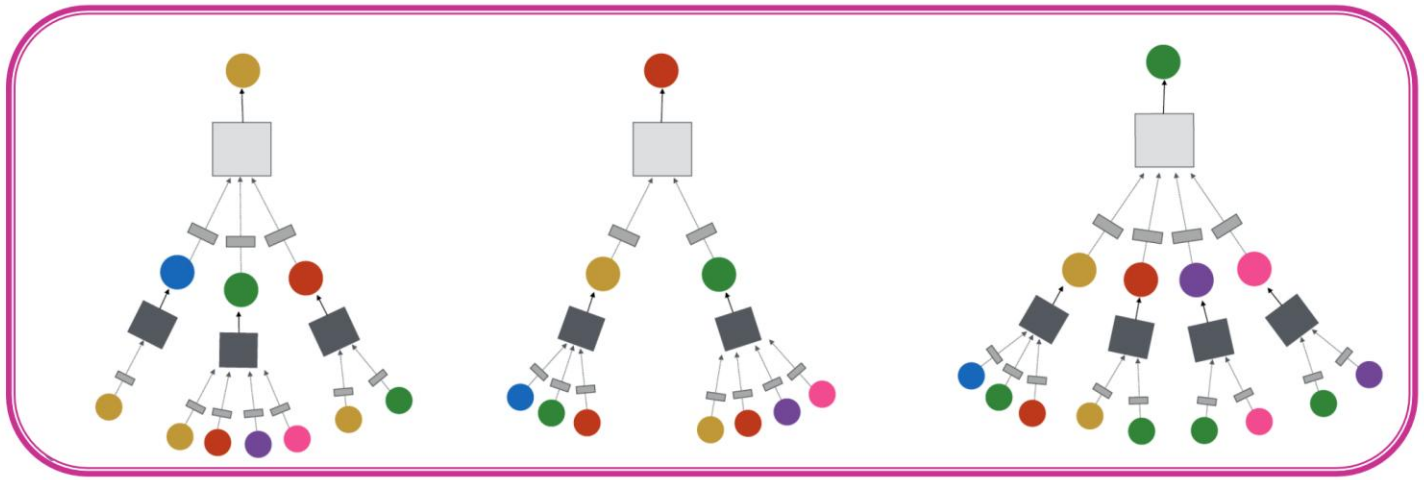


GNN Model Overview

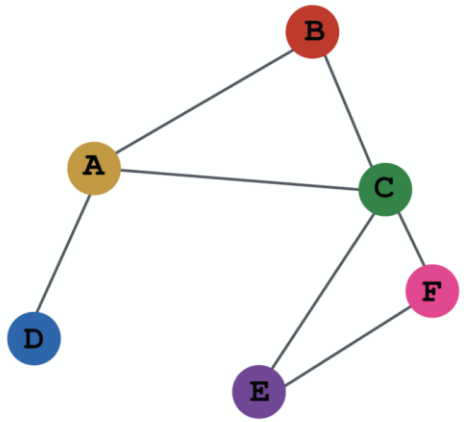


INPUT GRAPH

(3) Train on a set of nodes, i.e., a batch of compute graphs



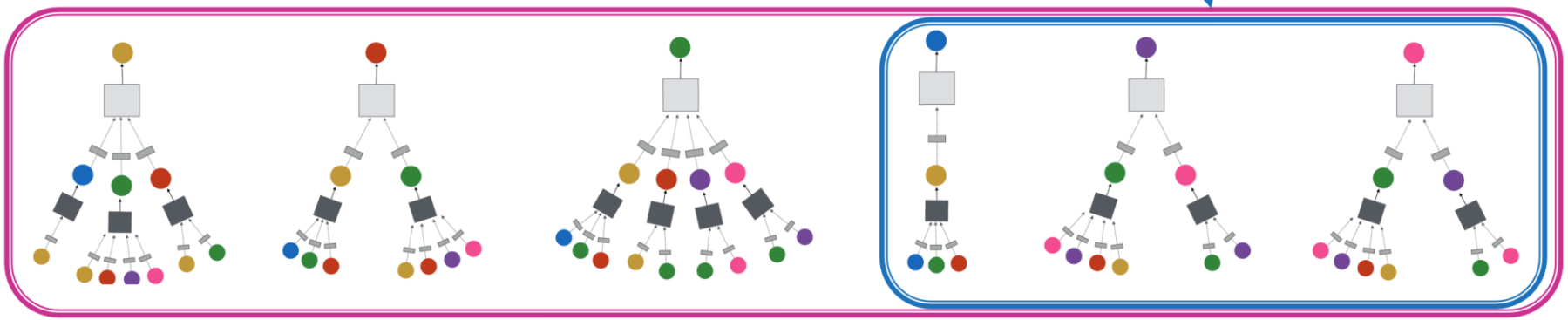
GNN Model Overview



INPUT GRAPH

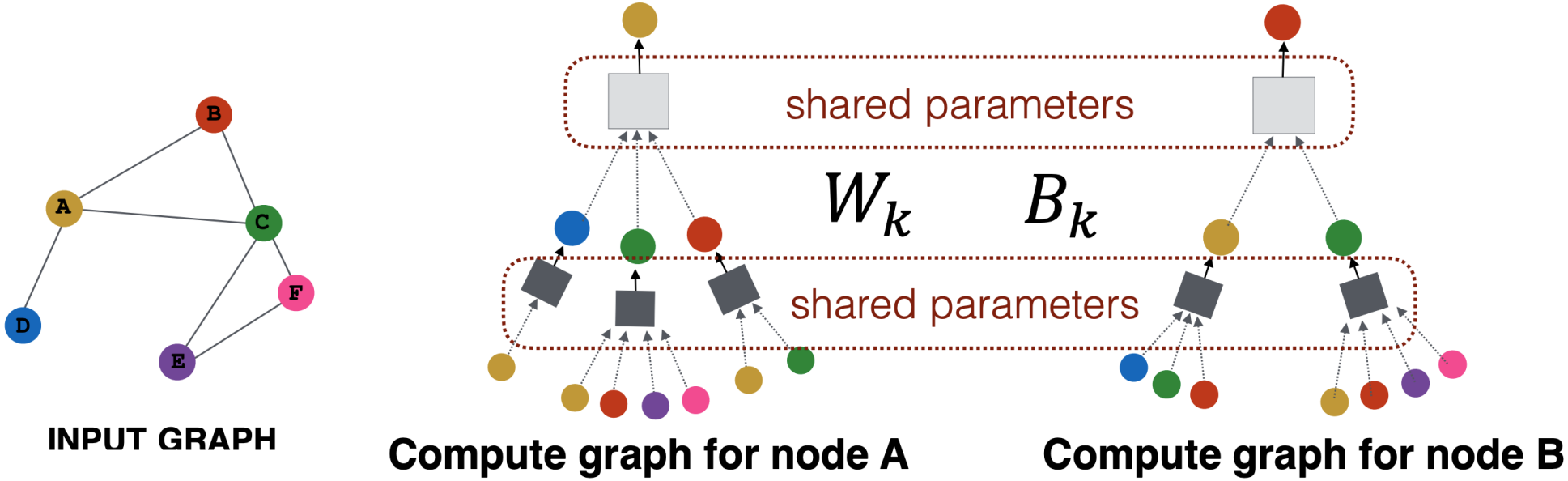
(4) Generate embeddings for nodes as needed

Even for nodes we never trained on!



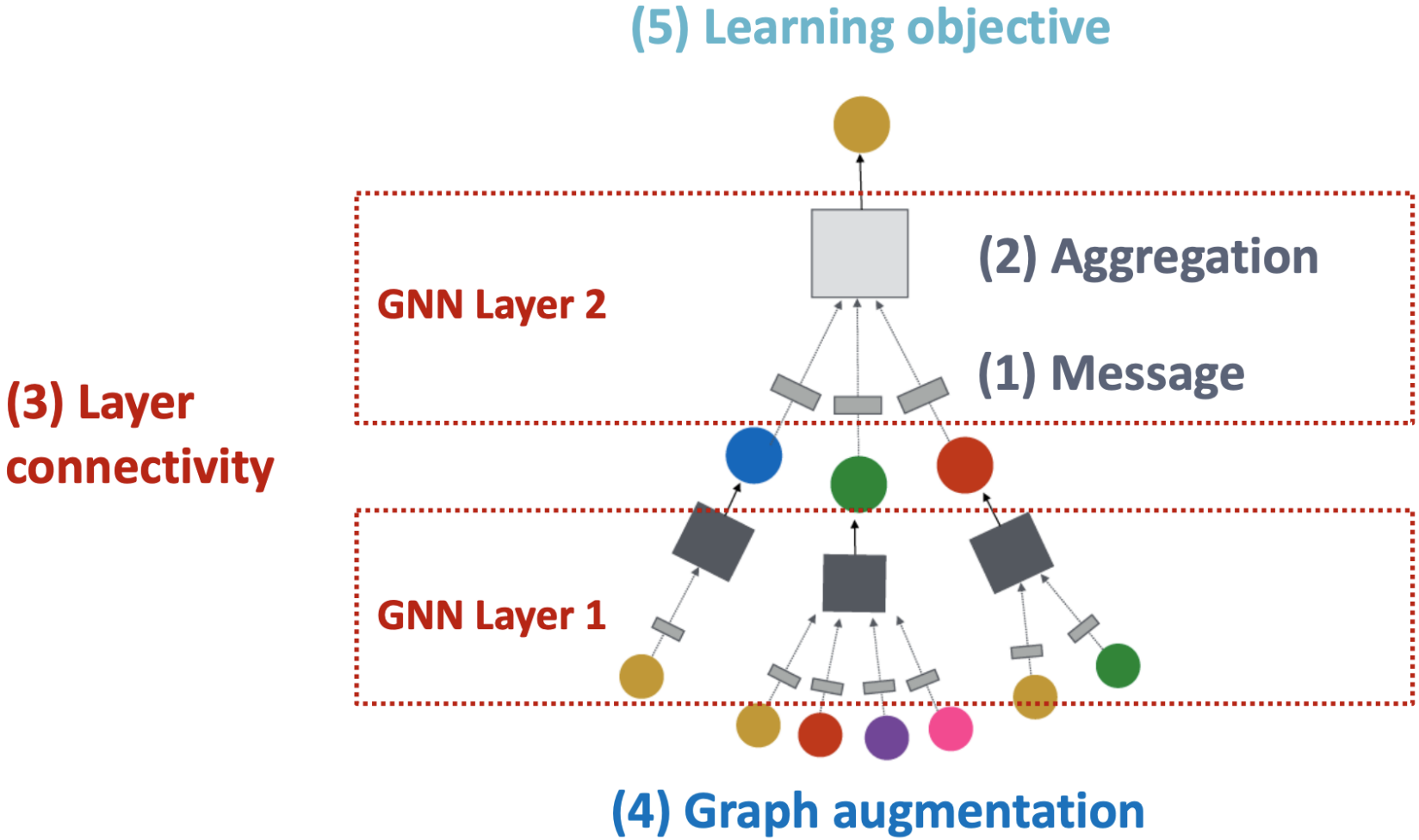
Generalize to unseen nodes

The same aggregation parameters are shared for all nodes



We can train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

General GNN Framework

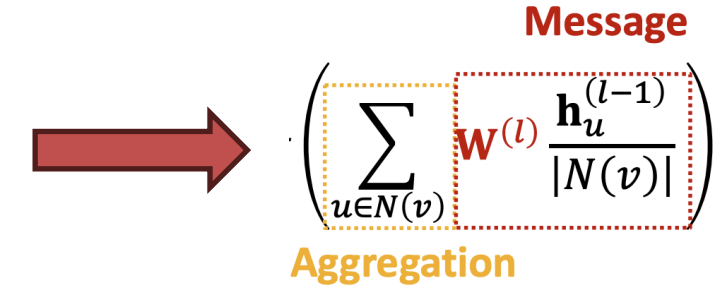


GCN fits the general framework

GNN layer = Message + Aggregation
Different instantiations under this perspective, such as GCN, GraphSAGE, GAT, …

For example, GCN:
In GCN the input graph is assumed to have self-edges that the node itself included in the summation.

$$\mathbf{W}^{(l)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}$$



GraphSAGE Layer

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{(l-1)}, \text{AGG}\left(\left\{\mathbf{h}_u^{(l-1)}, \forall u \in N(v)\right\}\right)\right)\right)$$

- **How to write this as Message + Aggregation?**
  - **Message** is computed within the $\text{AGG}(\cdot)$
  - **Two-stage aggregation**
    - **Stage 1:** Aggregate from node neighbors

$$\mathbf{h}_{N(v)}^{(l)} \leftarrow \text{AGG}\left(\left\{\mathbf{h}_u^{(l-1)}, \forall u \in N(v)\right\}\right)$$

    - **Stage 2:** Further aggregate over the node itself

$$\mathbf{h}_v^{(l)} \leftarrow \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}(\mathbf{h}_v^{(l-1)}, \mathbf{h}_{N(v)}^{(l)})\right)$$

Stage1 Choices

- **Mean:** Take a weighted average of neighbors

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}$$

Aggregation — Message computation

- **Pool:** Transform neighbor vectors and apply symmetric vector function $\text{Mean}(\cdot)$ or $\text{Max}(\cdot)$

$$\text{AGG} = \text{Mean}(\{\text{MLP}(\mathbf{h}_u^{(l-1)}), \forall u \in N(v)\})$$

Aggregation — Message computation

- **LSTM:** Apply LSTM to reshuffled of neighbors

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{(l-1)}, \forall u \in \pi(N(v))])$$

Aggregation

Graph Attention Networks

MACQUARIE University

For graph attention networks, we have

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Attention weights

How it differs to GCN/GraphSAGE?

In GCN/GraphSAGE, $\alpha_{vu} = \frac{1}{|N(v)|}$ is the weight.

Which means

All neighbors $u \in N(v)$ are equally important to node v

Graph Attention Networks



MACQUARIE University

However, **not all node's neighbors are equally important**

Attention is inspired by cognitive attention.
The attention α_{vu} should focuse on the important parts of the input data and fades out the rest.

Idea: the NN should devote more computing power on that small but important part of the data. Which part of the data is more important depends on the context and is learned through training.

Learning α_{vu}

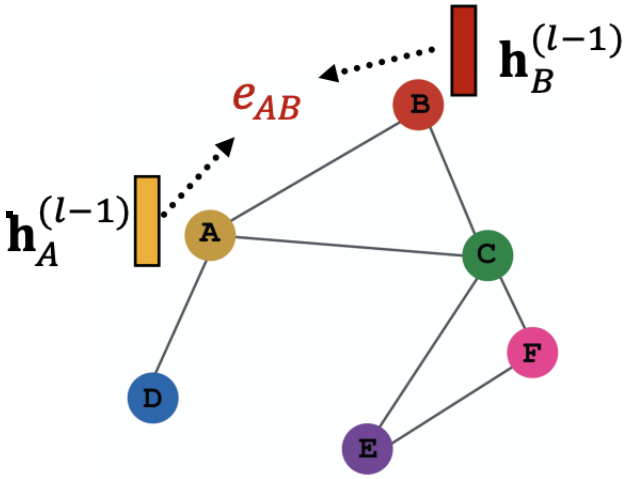
Let α_{vu} be computed as a byproduct of an attention mechanism a:

Attention mechanism a

- Compute attention coefficients e_{vu} across pairs of nodes u, v based on their messages:

$$e_{vu} = a(\mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)}\boldsymbol{h}_v^{(l-1)})$$

- e_{vu} indicates the importance of u's message to node v



$$e_{AB} = a(\mathbf{W}^{(l)}\mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)}\mathbf{h}_B^{(l-1)})$$

Learning α_{vu}

MACQUARIE University

Attention mechanism

- Normalize e_{vu} into the **final attention weight** α_{vu}
- We use **softmax** function so that $\sum_{u \in N(v)} \alpha_{vu} = 1$:

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

- Finally, we compute aggregation based on the final attention weight:

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

What is a

MACQUARIE University

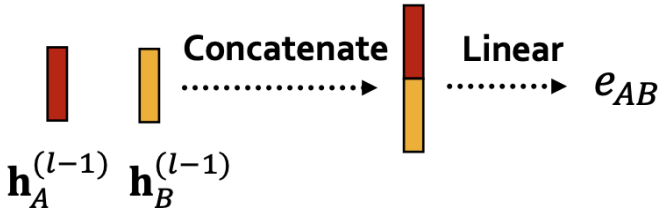
Attention mechanism

$$e_{vu} = a(\mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)}\boldsymbol{h}_v^{(l-1)})$$

?

The approach is agnostic to the choice of a

E.g., use a simple single-layer neural network



$$e_{AB} = a\left(\mathbf{W}^{(l)}\mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)}\mathbf{h}_B^{(l-1)}\right)$$
$$= \text{Linear}\left(\text{Concat}\left(\mathbf{W}^{(l)}\mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)}\mathbf{h}_B^{(l-1)}\right)\right)$$

Parameters of a are trained jointly:

Learn the parameters together with weight matrices (i.e., other parameter of the neural net $\mathbf{W}^{(l)}$) in an end-to-end fashion.

Multi-head attention

MACQUARIE University

Can stabilize the learning process of attention mechanism.

Create multiple attention scores (each replica with a different set of parameters):

$$\mathbf{h}_v^{(l)}[1] = \sigma\left(\sum_{u \in N(v)} \alpha_{vu}^{1} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

$$\mathbf{h}_v^{(l)}[2] = \sigma\left(\sum_{u \in N(v)} \alpha_{vu}^{2} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

$$\mathbf{h}_v^{(l)}[3] = \sigma\left(\sum_{u \in N(v)} \alpha_{vu}^{3} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Outputs are aggregated:

- By concatenation or summation
- $\mathbf{h}_v^{(l)} = \mathrm{AGG}(\mathbf{h}_v^{(l)}[1], \mathbf{h}_v^{(l)}[2], \mathbf{h}_v^{(l)}[3])$

Benefits of attention mechanism



MACQUARIE University

Key benefit: Allows for (implicitly) specifying different importance values to different neighbors

Computationally efficient:

- Computation of attentional coefficients can be parallelized across all edges of the graph
- Aggregation may be parallelized across all nodes

Storage efficient:

- Sparse matrix operations
- Fixed number of parameters, irrespective of graph size

Localized:

- Only attends over local network neighborhoods

Inductive capability:

- It is a shared edge-wise mechanism
- It does not depend on the global graph structure

Things to think about

How do GNNs compare to Convolutional Neural Nets?

Any issues with this current general framework?

